LAPORAN KEGIATAN UJIAN KOMPETENSI KEJURUAN
SMK MAHARDHIKA BATUJAJAR TAHUN PELAJARAN
2025/2026

Disusun Untuk Memenuhi Persyaratan kelulusan Ujian Kompetensi Keahlian
Rekayasa Perangkat Lunak Tahun Pelajaran
2025/2026

Disusun Oleh

Bintang Eka Wardhana Syarifudin NIS. 2324.10.387

PROGRAM KEAHLIAN PENGEMBANGAN PERANGKAT LUNAK DAN GIM KOMPETENSI KEAHLIAN REKAYASA PERANGKAT LUNAK
SMK MAHARDHIKA BATUJAJAR
2026

LAPORAN KEGIATAN UJI KOMPETENSI KEJURUAN SMK MAHARDHIKA

BATUJAJAR TAHUN PELAJARAN 2025-2026

KOMPETENSI KEAHLIAN	:	Rekayasa Perangkat Lunak
KELAS	:	XII RPL 2
NAMA LENGKAP	:	Bintang Eka Wardhana Syarifudin
NIS	:	2324.10.387
PAKET SOAL	:	2
JENIS APLIKASI	:	Website (Dashboard)
ALOKASI WAKTU	:	12 jam

1. PENJADWALAN KEGIATAN

Kegiatan	Minggu 1	Minggu 2	Minggu 3	Minggu 4	Minggu 5	Minggu 6
Analisis Soal	■					
Perancangan Sistem	■					
Perancangan DB	■	■				
Perancangan UI/UX	■	■				
Implementasi DB		■				
Implementasi UI/UX		■	■			
Implementasi Program			■	■		
Finalisasi Program				■		
Pengujian Program				■	■	
Hosting + Github					■	
Finalisasi Dokumentasi User Manual					■	
Finalisasi Projek UKK					■	■
Dokumentasi						■

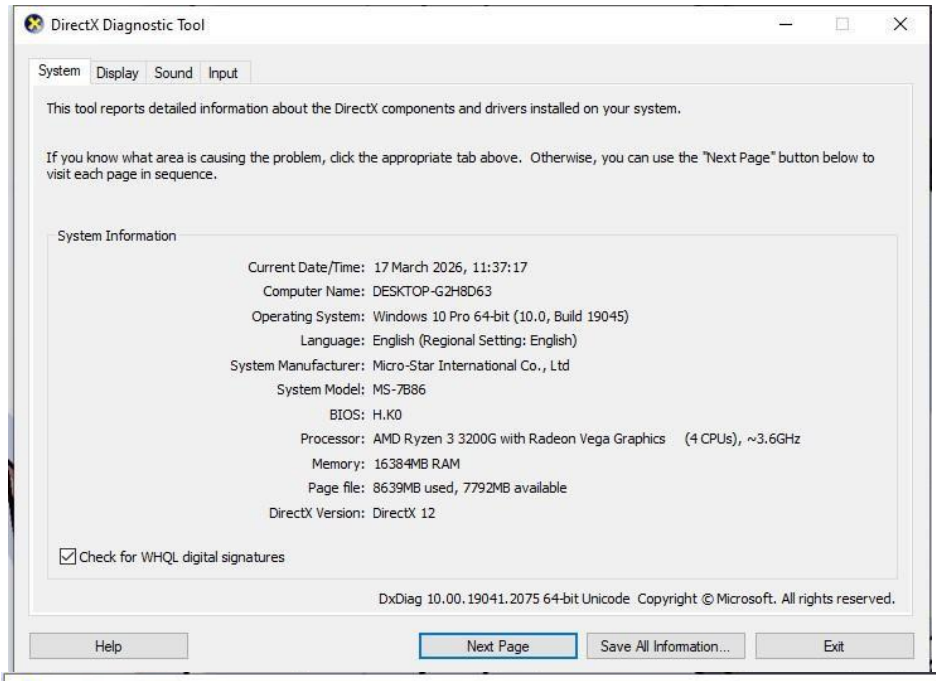
Keterangan: ■ = Minggu aktif pengerjaan

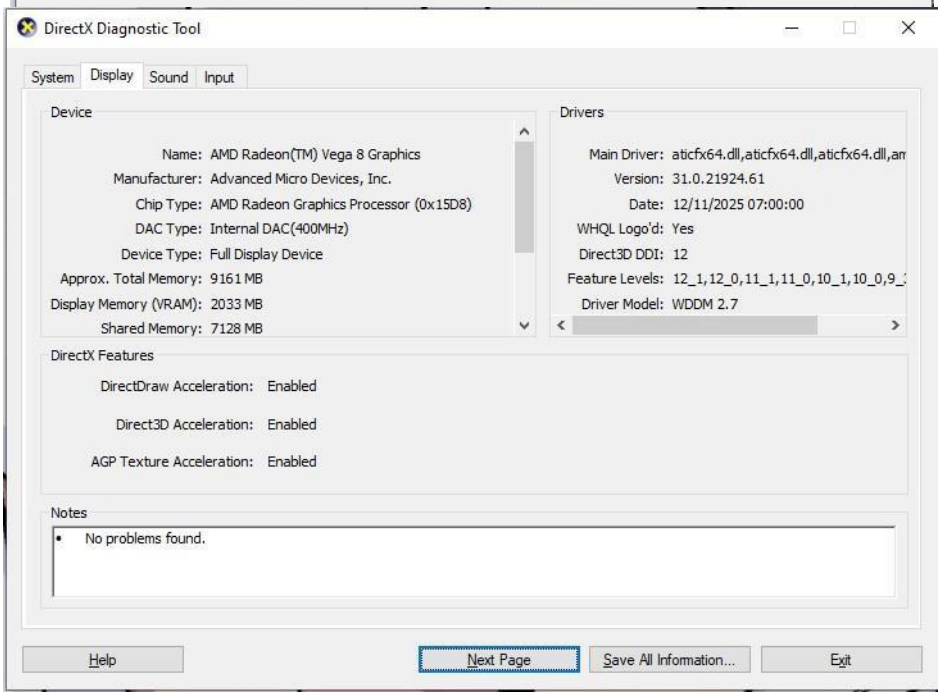
2. PERSIAPAN PERANGKAT KERAS

Berikut adalah spesifikasi perangkat keras yang digunakan dalam pengembangan Sistem Parkir Pintar Berbasis IoT:

A. Perangkat Komputer

Berikut adalah spesifikasi komputer yang digunakan dalam pengembangan dashboard website Sistem Parkir Pintar Berbasis IoT:





B. Perangkat Hardware IoT (Device)

Berikut adalah perangkat hardware yang digunakan untuk membangun prototype Sistem Parkir Pintar Berbasis IoT:

1. ESP32 Development Board (1 unit) — sebagai mikrokontroler utama yang menghubungkan komponen hardware ke internet melalui WiFi dan menjalankan protokol MQTT.
2. RFID Module Reader + Tag (2 unit) — sebagai pembaca kartu RFID kendaraan di gerbang Entry dan gerbang Exit.
3. Motor Servo (2 unit) — sebagai penggerak palang gerbang Entry dan gerbang Exit yang dikontrol oleh ESP32.
4. OLED 128x64 (1 unit) — sebagai display yang menampilkan pesan kepada pengguna saat check-in dan check-out parkir.
5. Kabel Jumper (1 set) — sebagai penghubung antarkomponen pada rangkaian hardware.
6. Kabel USB Type C (1 unit) — untuk pemrograman dan pengisian daya ESP32 Development Board.

Gambar 2.1 ESP32 Development Board

Gambar 2.2 RFID Module Reader + Tag

Gambar 2.3 Motor Servo

Gambar 2.4 OLED 128x64

Gambar 2.5 Kabel Jumper

Gambar 2.6 Kabel USB Type C

Tabel Koneksi Wiring ESP32

Berikut adalah tabel koneksi pin antara ESP32 Development Board dengan seluruh komponen hardware yang digunakan:

Tabel 2.1 Koneksi Wiring ESP32 dengan Komponen Hardware

Pin ESP32	RFID Entry (MFRC522)	RFID Exit (MFRC522)	Komponen Lain
GPIO18	SCK	SCK	—
GPIO23	MOSI	MOSI	—
GPIO19	MISO	MISO	—
GPIO5	SDA/SS	—	—
GPIO27	RST	—	—
GPIO15	—	SDA/SS	—
GPIO17	—	RST	—
GPIO32	—	—	Servo Entry (DATA)
GPIO33	—	—	Servo Exit (DATA)
GPIO21	—	—	OLED SSD1306 (SDA)
GPIO22	—	—	OLED SSD1306 (SCL)
GND	GND	GND	GND Servo Entry, Servo Exit, OLED
3.3V	VCC	VCC	VCC OLED SSD1306
5V (VIN)	—	—	VCC Servo Entry, Servo Exit

3. PERSIAPAN PERANGKAT LUNAK

Pembuatan aplikasi dashboard dan konfigurasi perangkat IoT membutuhkan beberapa perangkat lunak berikut:

1. Visual Studio Code

Visual Studio Code digunakan sebagai kode editor utama untuk penulisan kode HTML, PHP, CSS, dan JavaScript dalam pembuatan dashboard website .

2. SUPABASE

Supabase digunakan sebagai backend-as-a-service yang menyediakan database, autentikasi, dan API secara otomatis, sehingga dashboard dapat dijalankan dan diuji tanpa perlu menyiapkan server lokal secara manual.

3. Google Chrome

Google Chrome digunakan sebagai web browser untuk membuka dan menguji tampilan dashboard website, serta mengakses Supabase.

4. MQTTX

MQTTX digunakan sebagai aplikasi MQTT Client untuk menguji komunikasi antara device ESP32 dan dashboard melalui broker MQTT (broker.hivemq.com) dengan topik-topik yang telah ditentukan.

5. Astah Professional

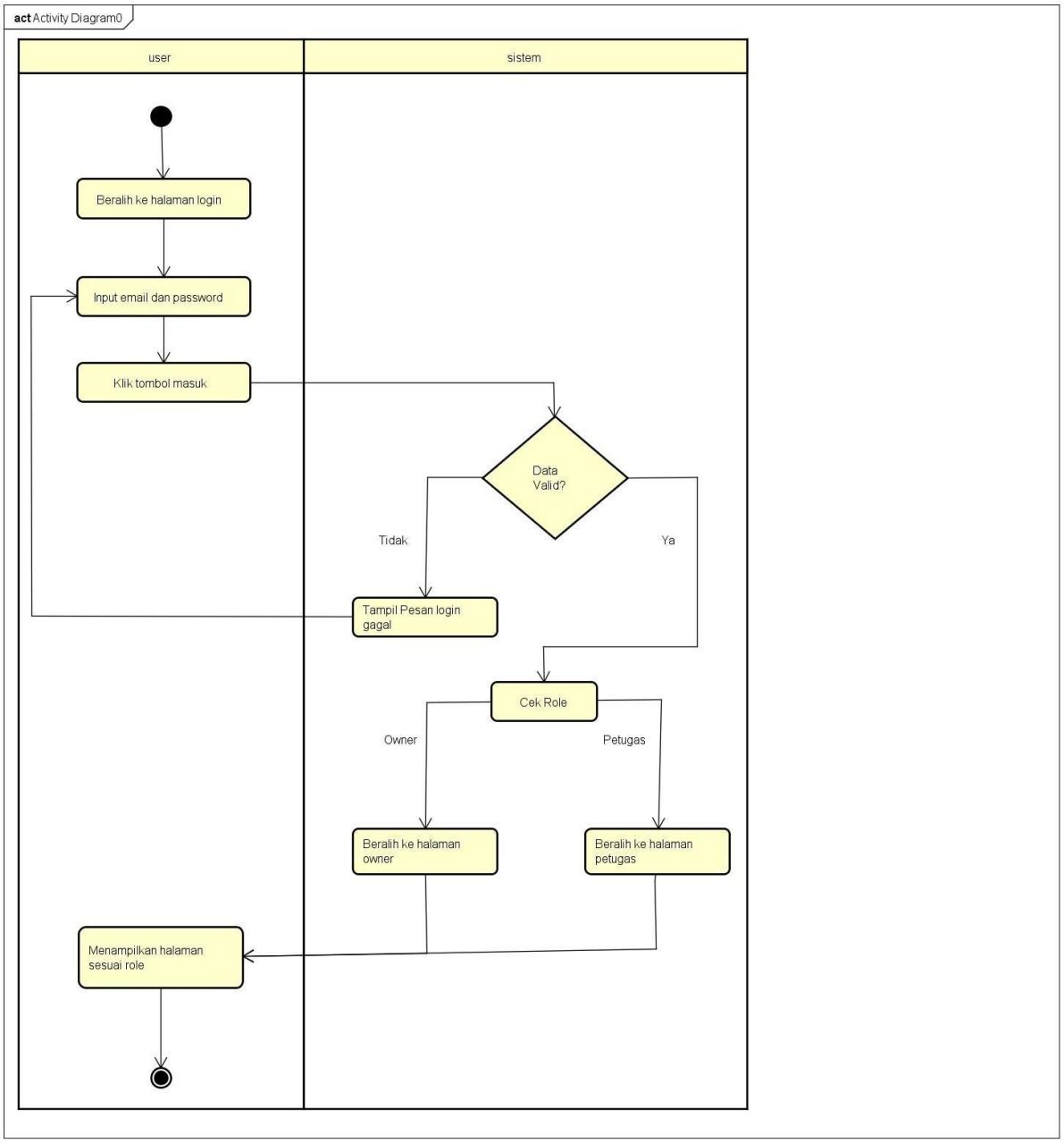
Astah Professional digunakan untuk perancangan diagram aplikasi dalam bentuk Activity Diagram, Use Case Diagram, dan Sequence Diagram agar alur sistem mudah dipahami.

6. Balsamiq Wireframes

Balsamiq digunakan untuk membuat mockup tampilan antarmuka dashboard secara visual sebelum diimplementasikan ke dalam kode program.

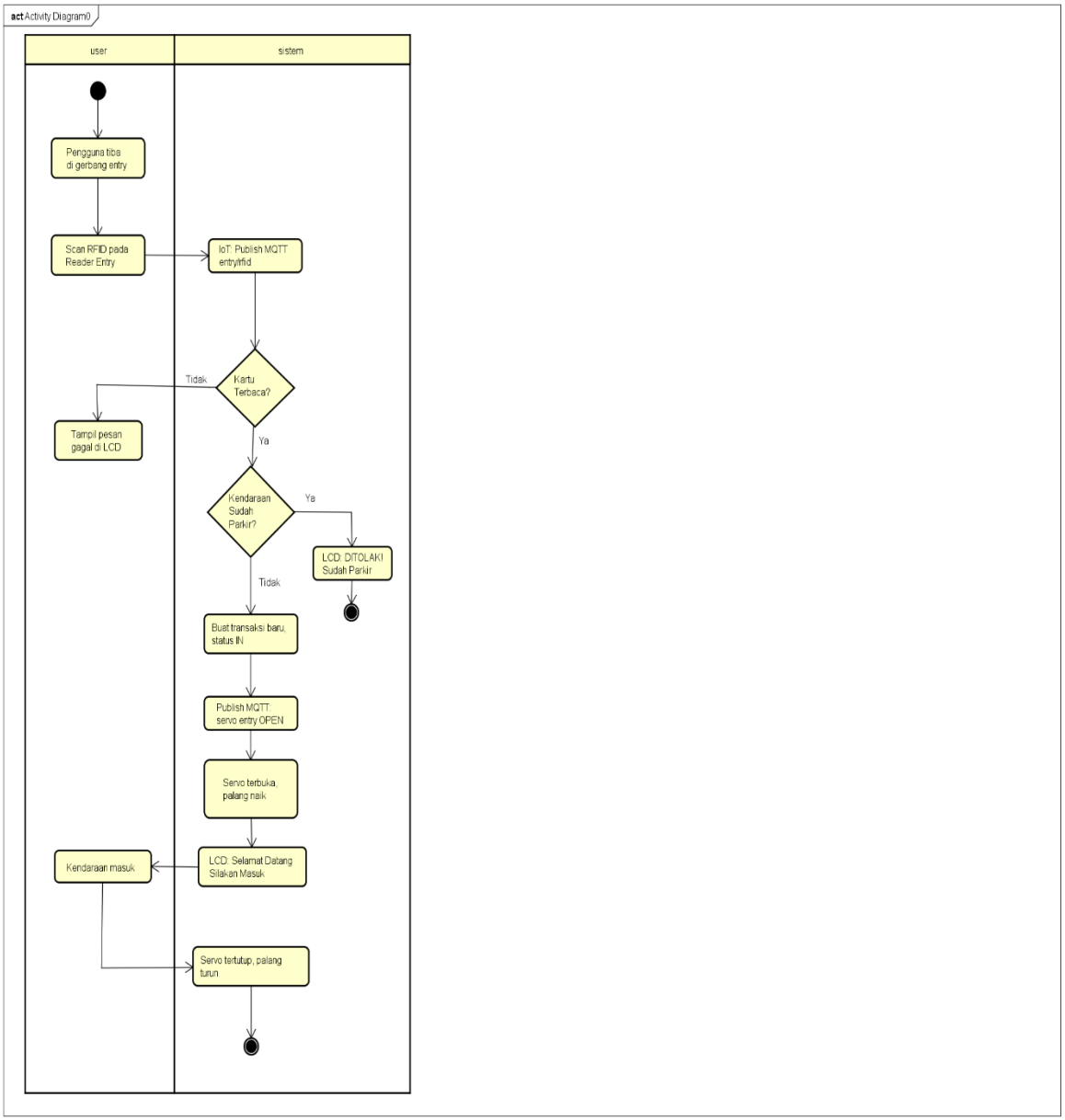
4. PERANCANGAN ALGORITMA APLIKASI

Activity Diagram Login



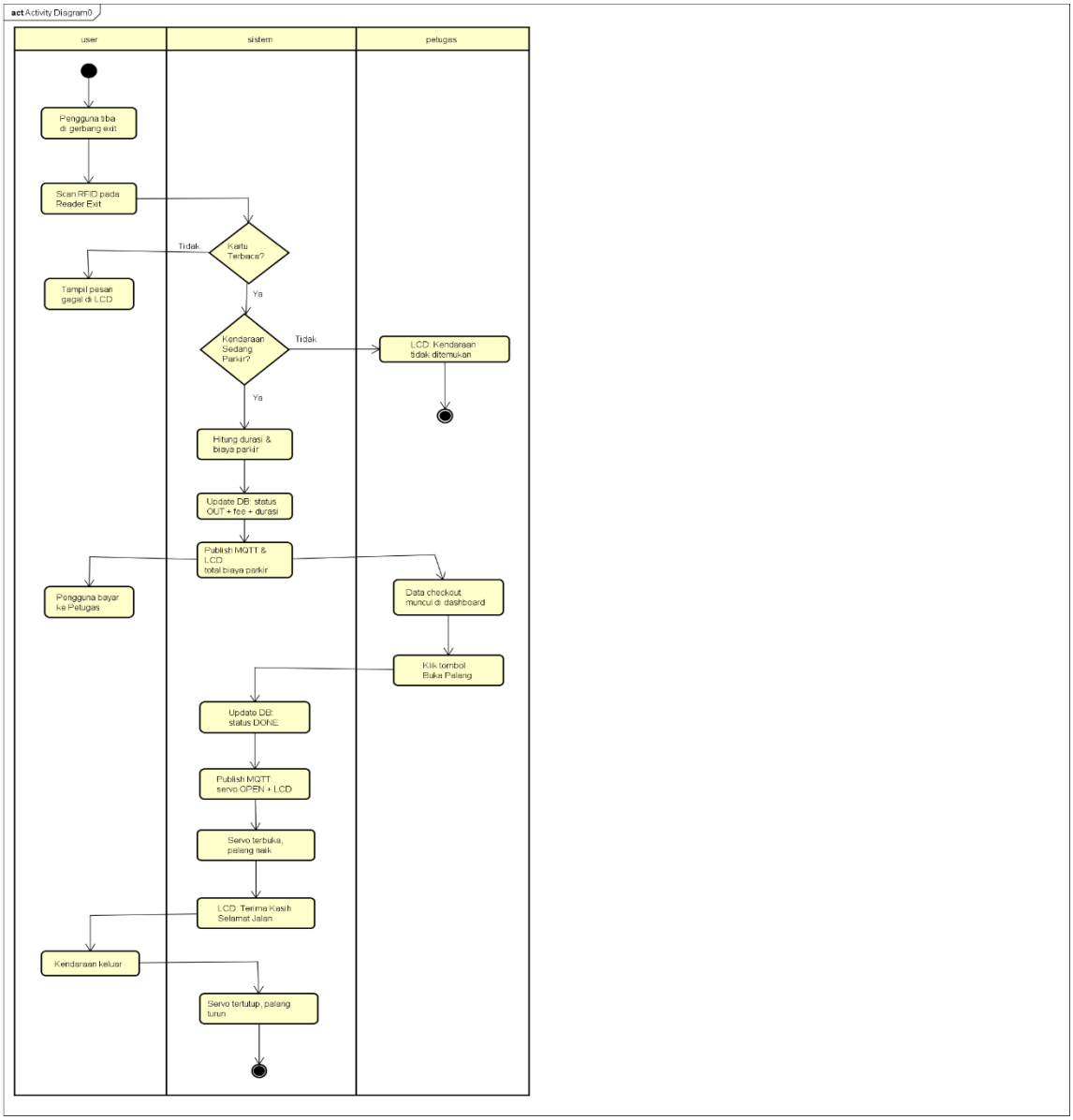
Gambar 4.1 Activity Diagram Login

Activity Diagram Check-In Kendaraan



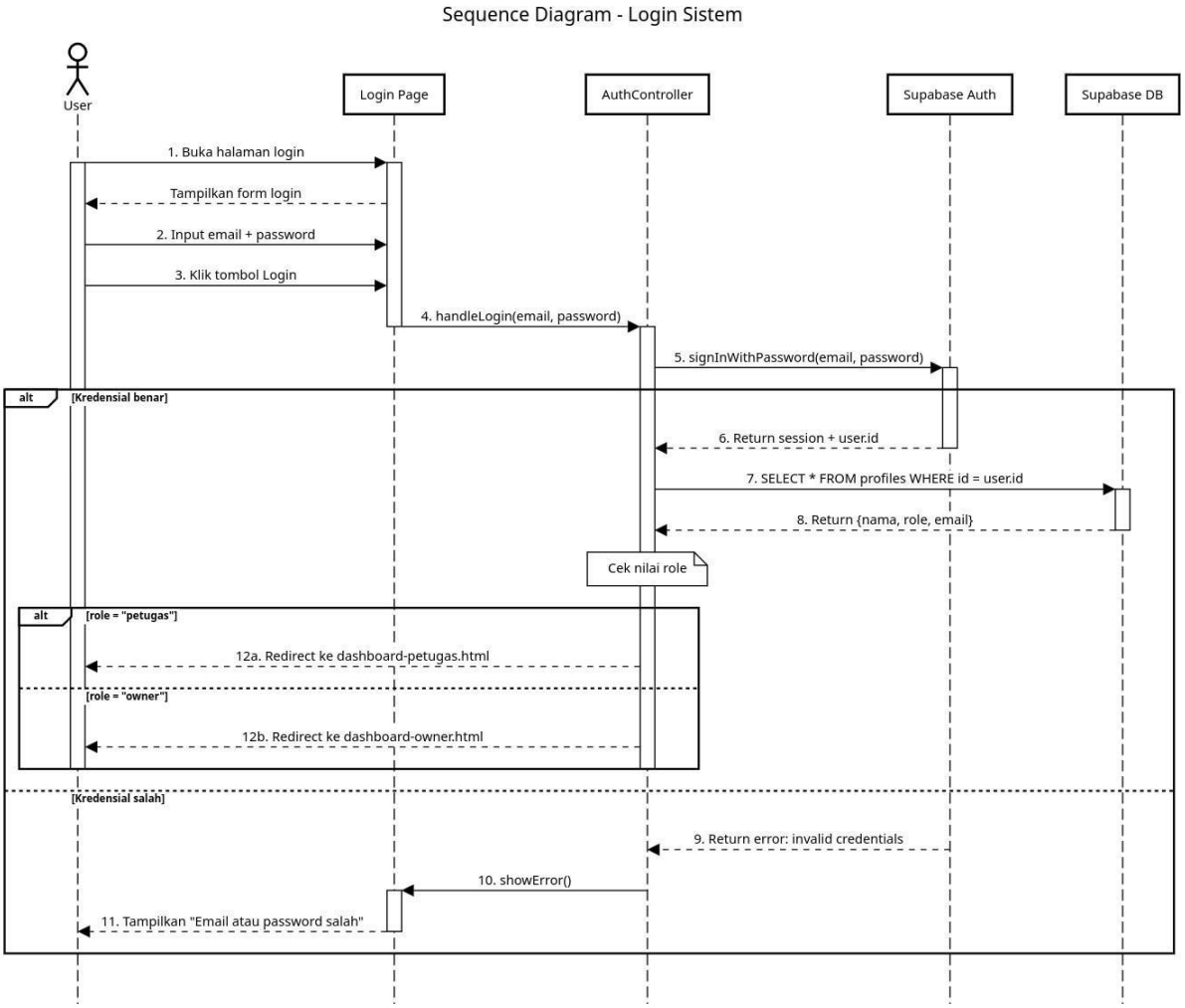
Gambar 4.2 Activity Diagram Check-In Kendaraan

Activity Diagram Check-Out Kendaraan



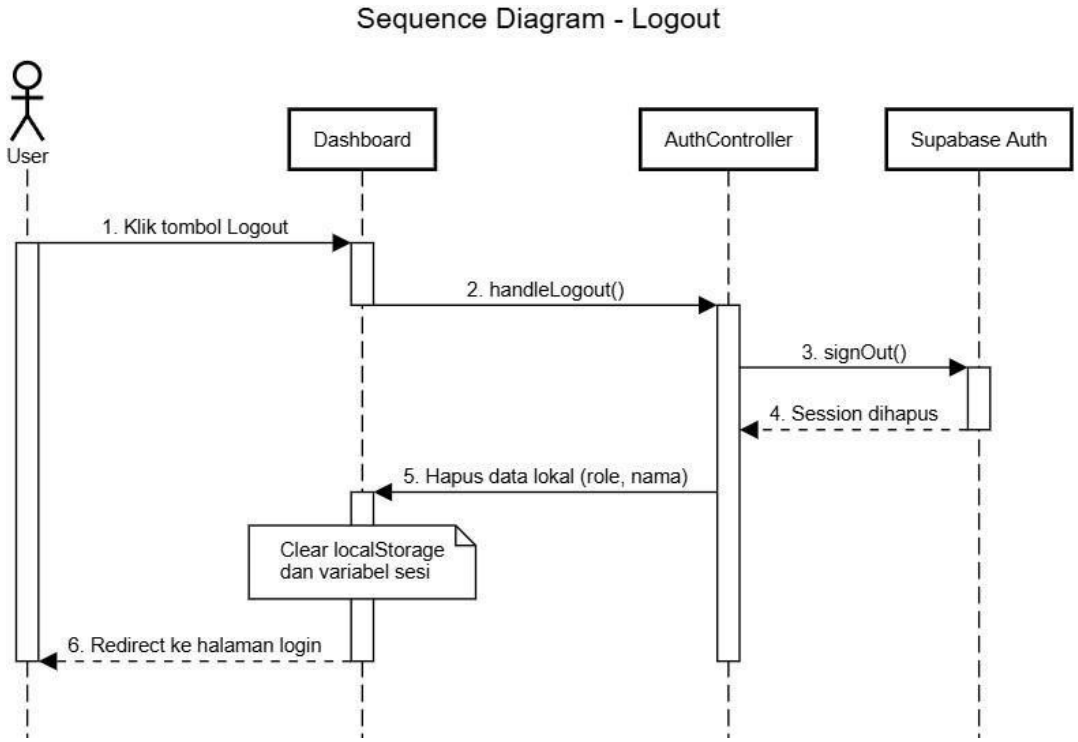
Gambar 4.3 Activity Diagram Check-Out Kendaraan

Sequence Diagram Login



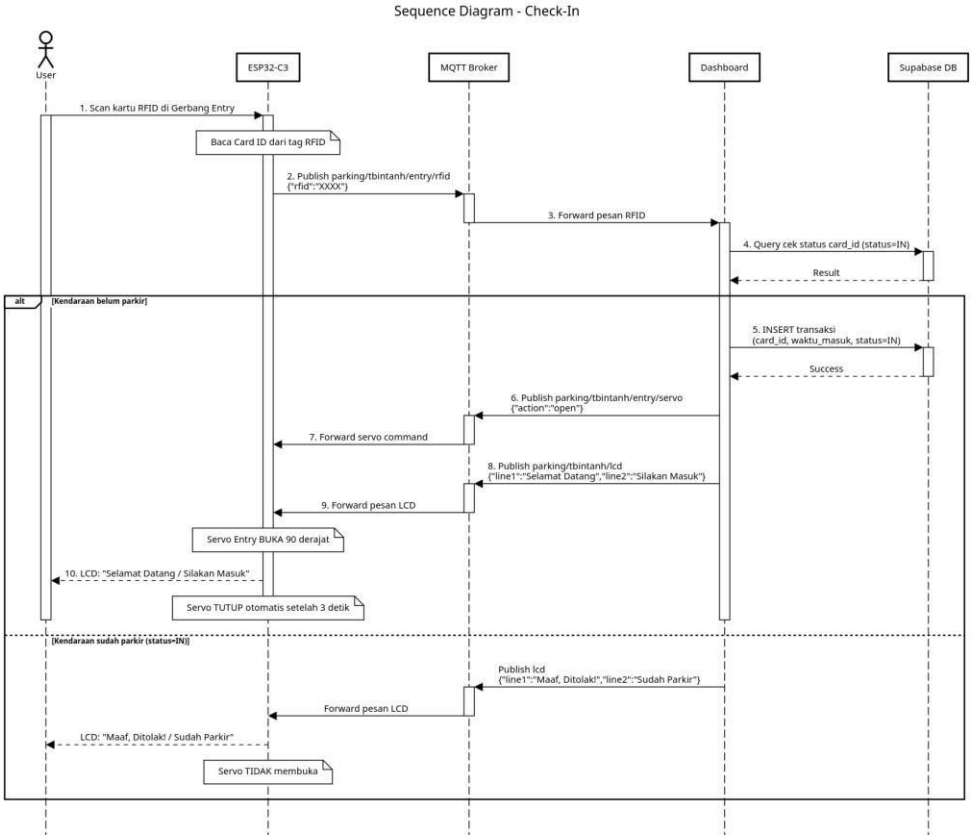
Gambar 4.4 Sequence Diagram Login

Sequence Diagram Logout



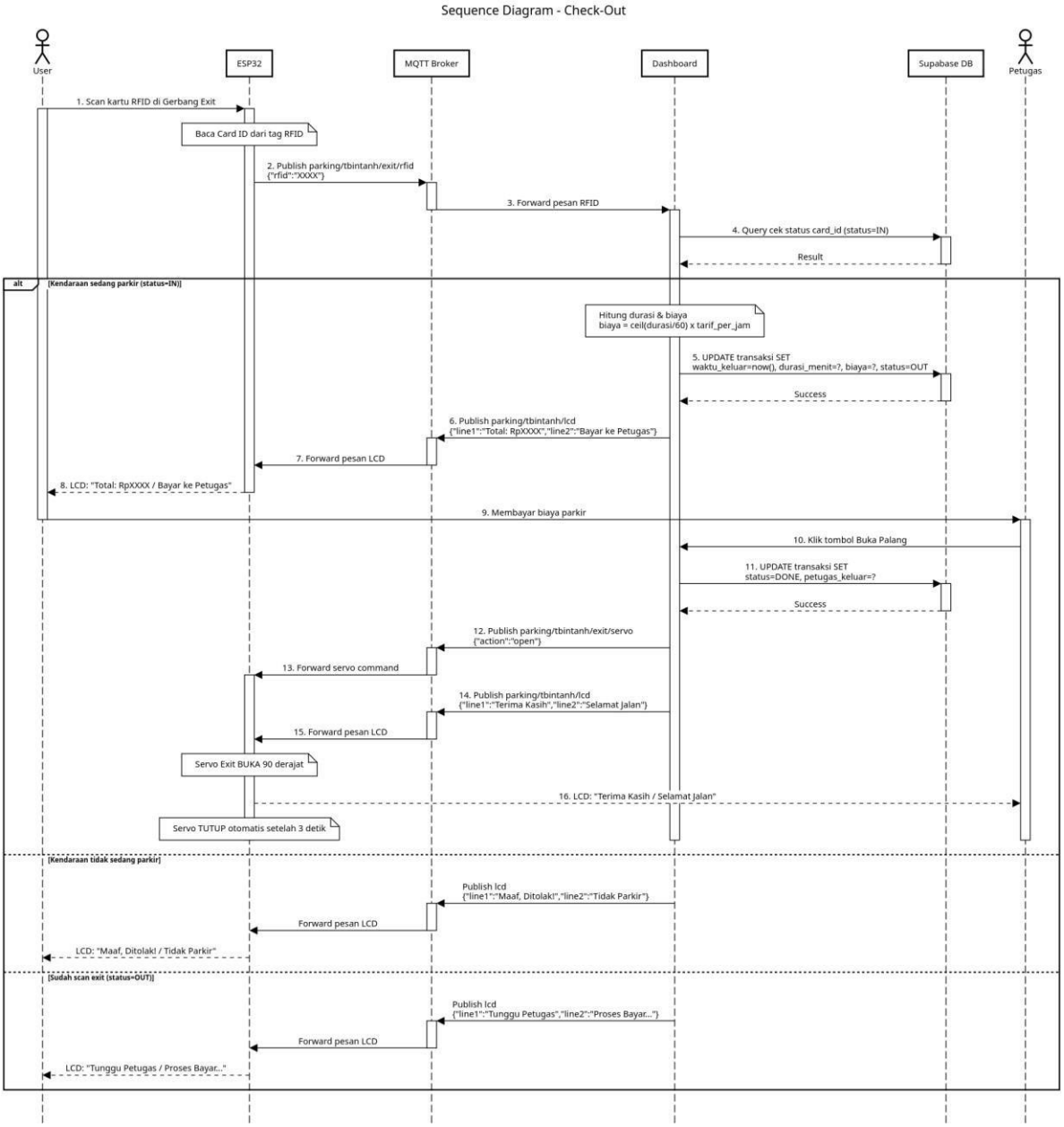
Gambar 4.5 Sequence Diagram Logout

Sequence Diagram Check-In



Gambar 4.6 Sequence Diagram Check-In Kendaraan

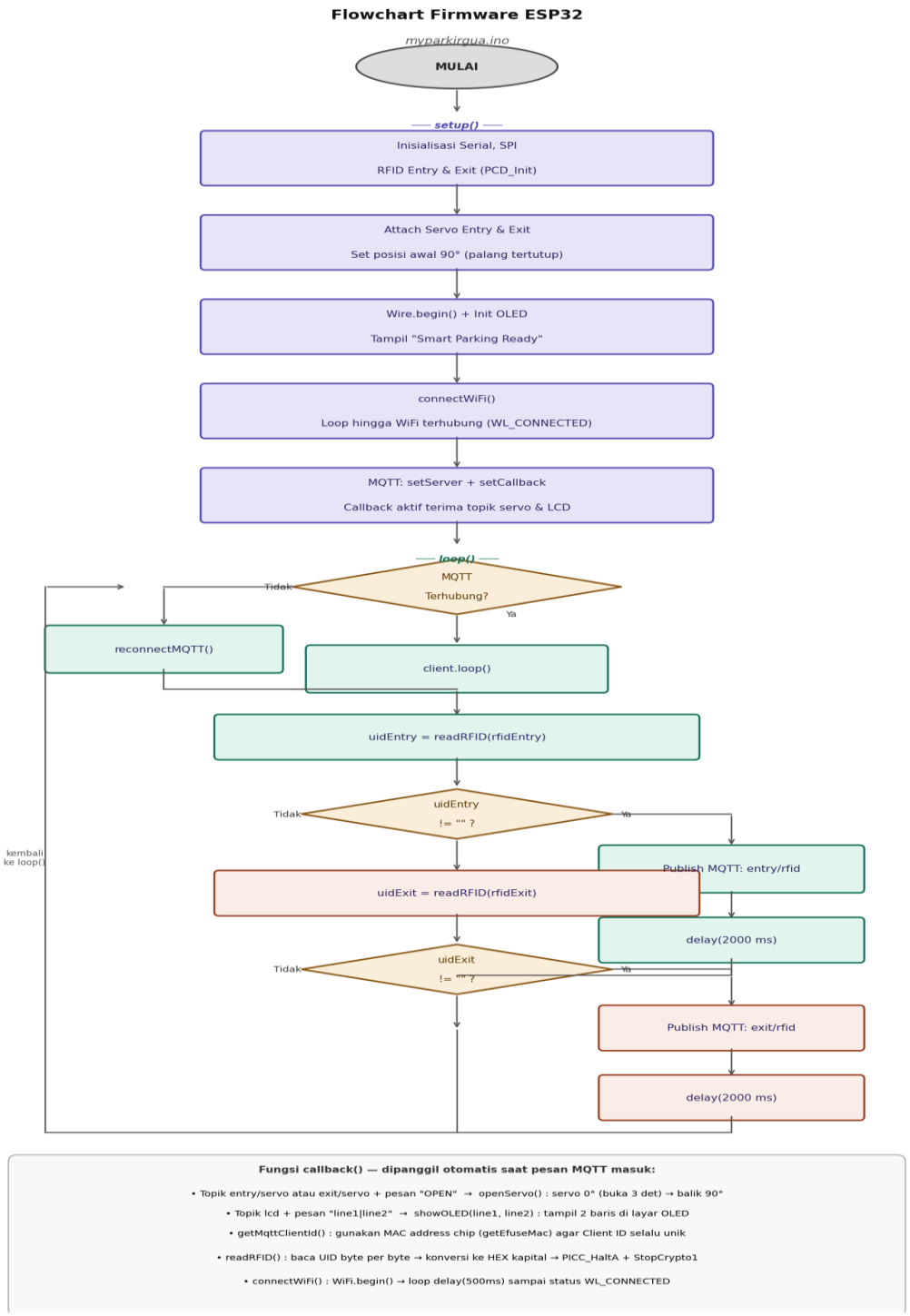
Sequence Diagram Check-Out



Gambar 4.7 Sequence Diagram Check-Out Kendaraan

Flowchart Firmware Device (ESP32)

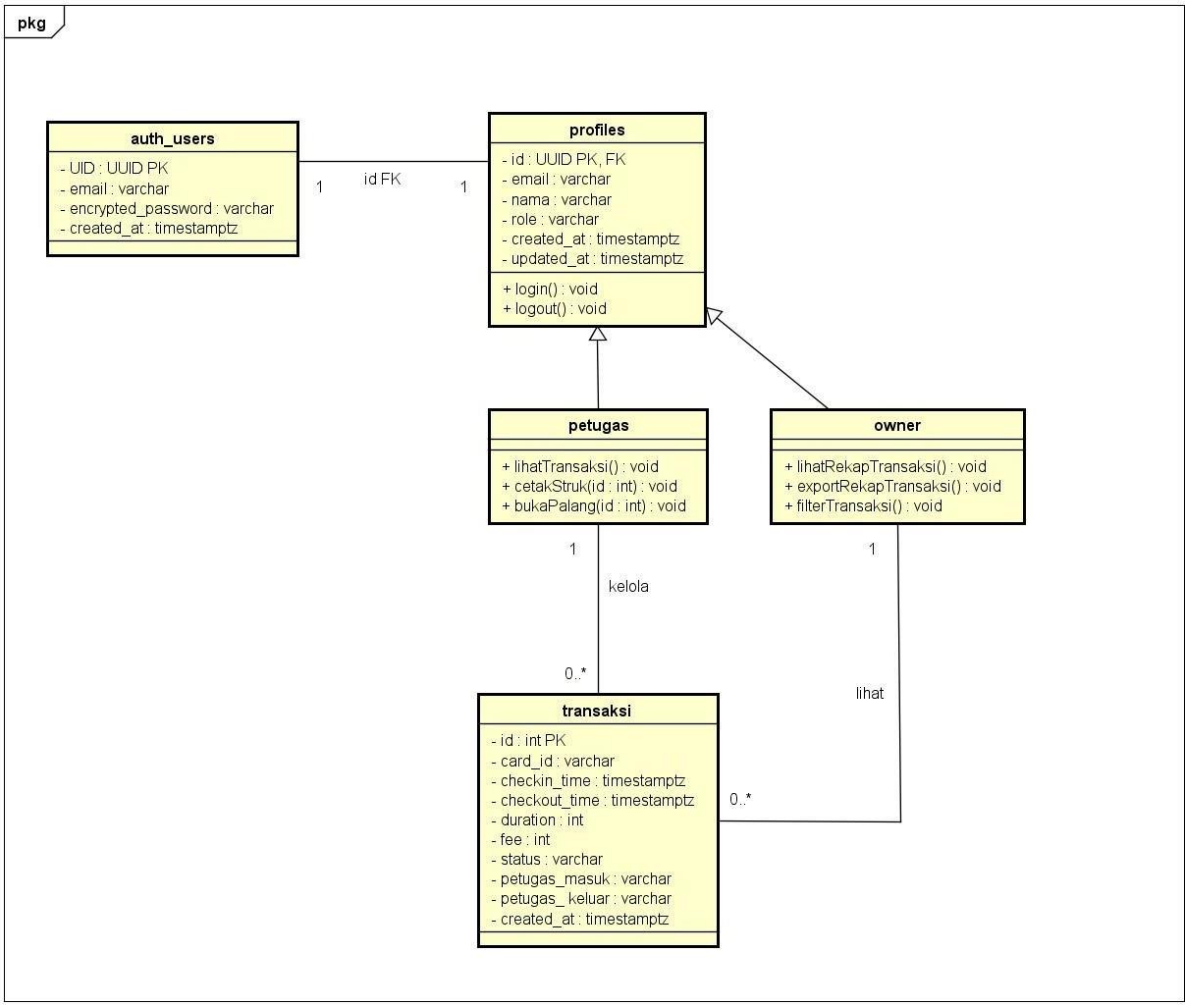
Berikut adalah flowchart alur program firmware ESP32 (myparkirgua.ino) mulai dari proses inisialisasi pada fungsi setup() hingga eksekusi berulang pada fungsi loop():



Gambar 4.8 Flowchart Firmware ESP32 – myparkirgua.ino

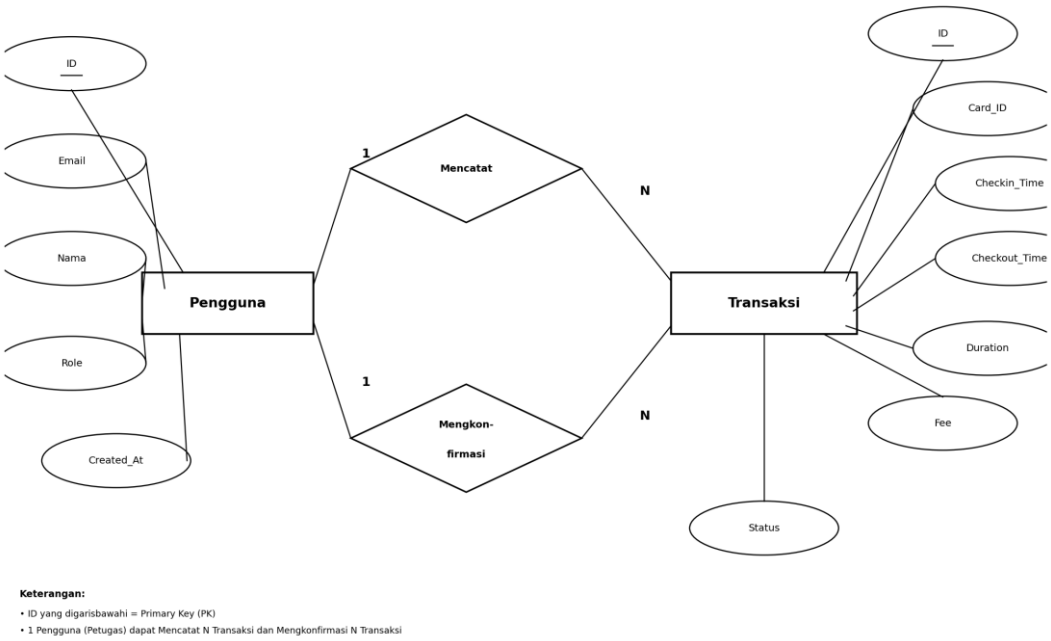
5. PERANCANGAN BASIS DATA

Berikut adalah perancangan basis data (Class Diagram) Sistem Parkir Pintar Berbasis IoT:



Gambar 5.1 Class Diagram Sistem Parkir Pintar Berbasis IoT

Berikut adalah Entity Relationship Diagram (ERD) yang menggambarkan relasi antar entitas pada basis data Sistem Parkir Pintar Berbasis IoT:



Gambar 5.2 Entity Relationship Diagram (ERD) Sistem Parkir Pintar Berbasis IoT

6. DUMMY DATA

Tabel transaksi

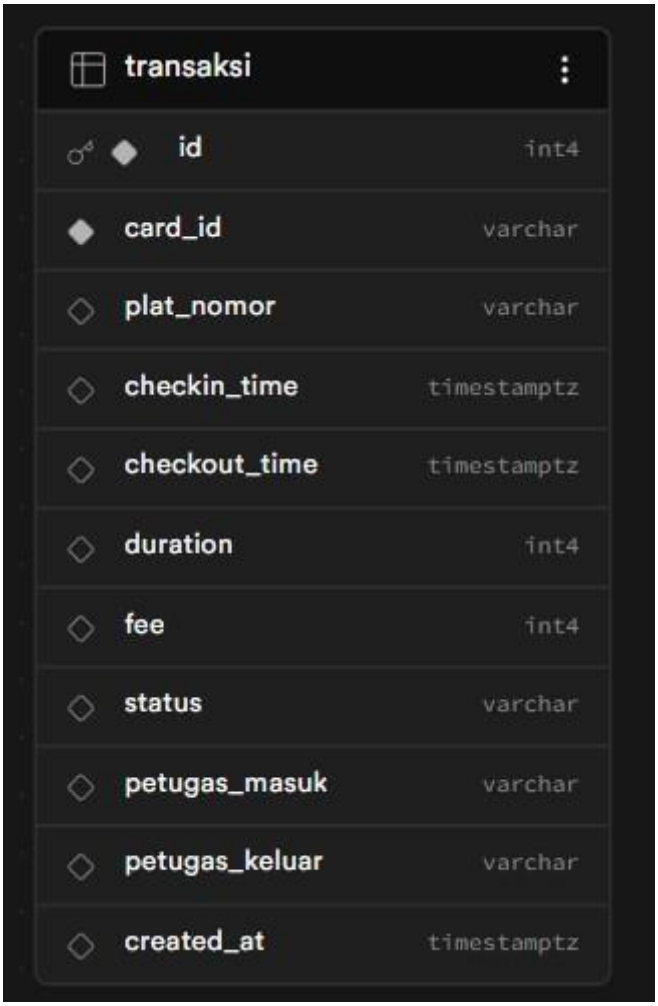
id	card_id	checkin_time	checkout_time	duration	fee	status
1	A1B2C3D4	2026-03-17 08:00:00	2026-03-17 10:00:00	120	4000	DONE
2	E5F6G7H8	2026-03-17 09:30:00	NULL	NULL	NULL	IN
3	I9J0K1L2	2026-03-17 10:15:00	2026-03-17 10:45:00	30	2000	OUT

Tabel profiles

id	email	nama	role	created_at
0a9e386aed14-42f7a03d-be2b7591fe87	owner@test.com	Owner Parkir	owner	2026-02-09 01:18:33
cb663b97-7a98-42f6-8da017c2909bbb9c	petugas@test.com	Petugas Parkir	petugas	2026-02-09 01:18:16

7. PEMBUATAN BASIS DATA

Tabel transaksi



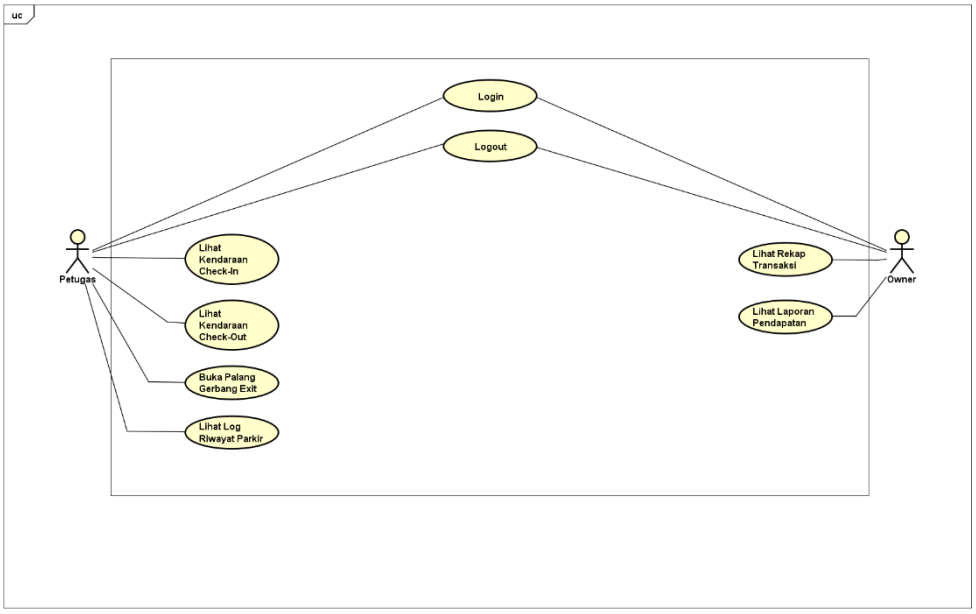
Gambar 7.1 Struktur Tabel transaksi di Supabase

Tabel profiles

Gambar 7.2 Struktur Tabel profiles di Supabase

8. PERANCANGAN TAMPILAN ANTARMUKA

Use Case Diagram



Gambar 8.1 Use Case Diagram Sistem Parkir Pintar Berbasis IoT

Use Case Scenario Check-In Kendaraan

Aktor	:	Sistem (Device ESP32 RFID Entry)
Deskripsi	:	Kendaraan melakukan scan tag RFID di gerbang masuk untuk melakukan check-in parkir.

Pengguna / Device	Sistem (Dashboard)
1. Kendaraan mendekatkan tag RFID ke reader gerbang Entry.	2. Membaca card_id dari tag RFID.
	3. Memeriksa apakah card_id tersebut sedang parkir (status IN).
	4a. Jika sudah parkir: menampilkan pesan 'Kendaraan Sudah Parkir' di LCD. Proses berhenti.
	4b. Jika belum parkir: menyimpan data check-in ke database (status=IN).
	5. Mengirim perintah buka palang Entry via MQTT topik servo Entry.
	6. Menampilkan pesan 'Selamat Datang Silakan Masuk' di OLED via MQTT topik LCD.

Use Case Scenario Check-Out Kendaraan

Aktor	:	Sistem (Device ESP32 RFID Exit)
Deskripsi	:	Kendaraan melakukan scan tag RFID di gerbang keluar untuk melakukan check-out parkir.

Pengguna / Device	Sistem (Dashboard)
1. Kendaraan mendekatkan tag RFID ke reader gerbang Exit.	2. Membaca card_id dari tag RFID.
	3. Memeriksa apakah card_id tersebut sedang parkir (status IN).
	4a. Jika tidak sedang parkir: menampilkan 'Kendaraan Tidak Ditemukan' di LCD.
	4b. Jika sedang parkir: menghitung durasi parkir dan total biaya (Rp 2.000/jam).
	5. Memperbarui data transaksi (checkout_time, duration, fee, status=OUT).
	6. Menampilkan total biaya parkir di LCD, menunggu petugas membuka palang.

Use Case Scenario Buka Palang Exit (Konfirmasi Pembayaran)

Aktor	:	Petugas
Deskripsi	:	Petugas membuka palang keluar melalui tombol aksi di dashboard setelah kendaraan membayar.

Pengguna / Device	Sistem (Dashboard)
1. Menekan tombol 'Buka Palang' pada kendaraan berstatus OUT di dashboard.	2. Memeriksa status transaksi di database.
	3. Mengirim perintah buka palang ke servo Exit melalui topik MQTT.
	4. Menampilkan 'Terima Kasih Selamat Jalan' di LCD via topik MQTT LCD.
	5. Memperbarui status transaksi menjadi DONE di database.

Use Case Scenario Lihat Data Parkir Aktif & Check-Out

Aktor	:	Petugas
Deskripsi	:	Petugas melihat daftar kendaraan yang sedang parkir dan kendaraan yang akan keluar.

Pengguna / Device	Sistem (Dashboard)
1. Membuka halaman utama dashboard.	2. Menampilkan daftar transaksi status IN (parkir aktif) dan status OUT (menunggu konfirmasi) dengan tombol aksi.

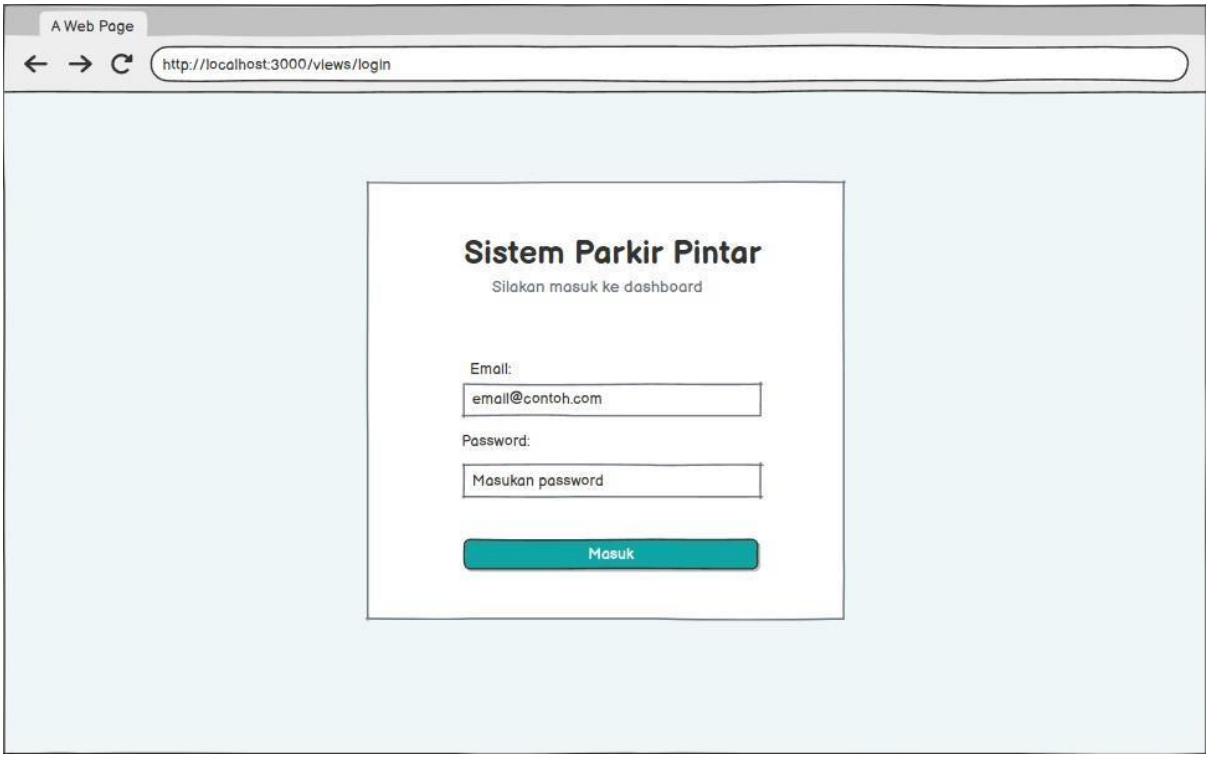
Use Case Scenario Lihat Log Riwayat Transaksi

Aktor	:	Petugas / Owner
Deskripsi	:	Melihat riwayat seluruh transaksi kendaraan yang sudah selesai parkir.

Pengguna / Device	Sistem (Dashboard)
1. Masuk ke dashboard petugas / owner	2. Menampilkan seluruh transaksi dengan status DONE beserta detail durasi dan biaya.

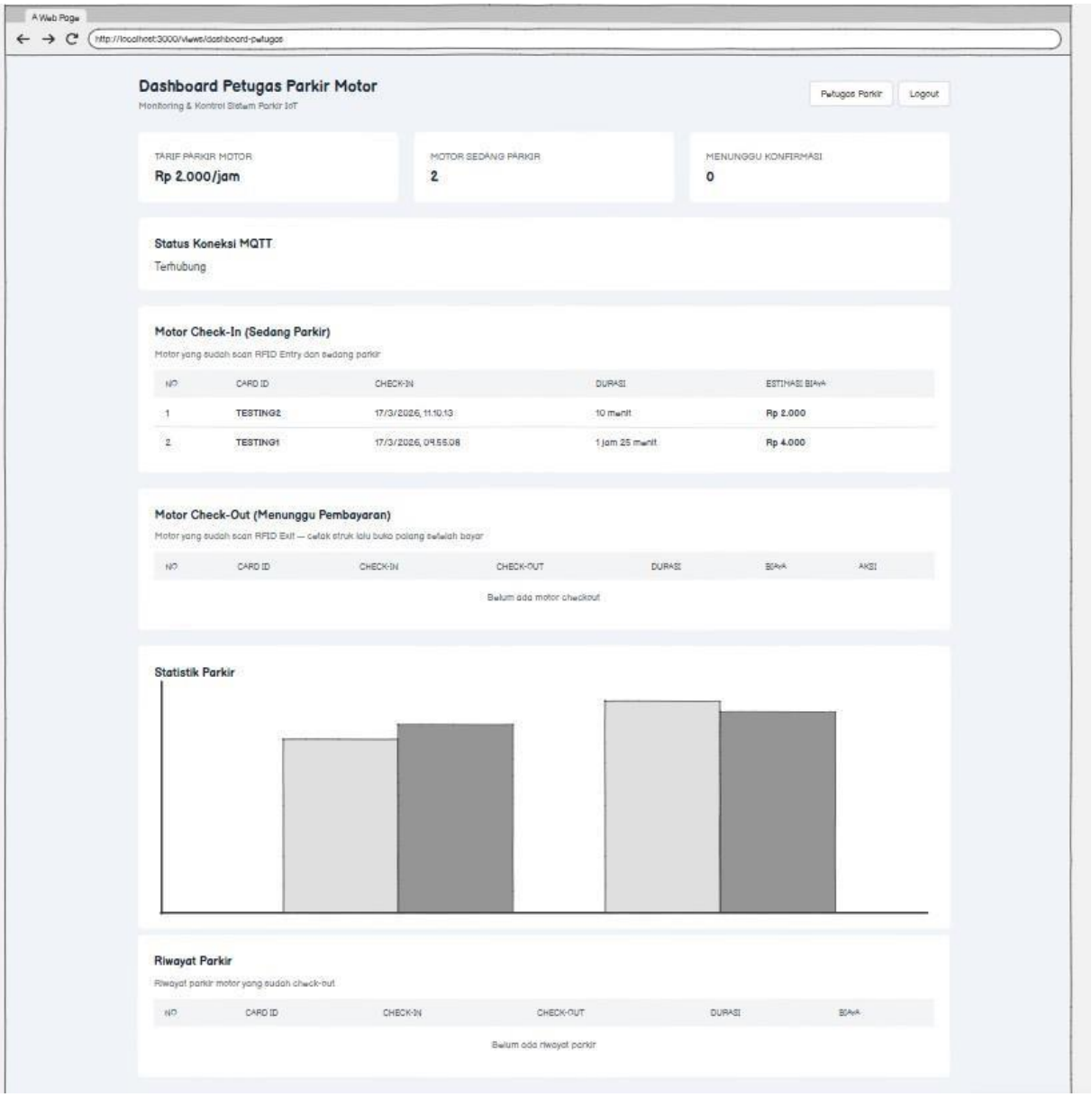
Perancangan Antarmuka (Mock Up)

Mockup Halaman Login



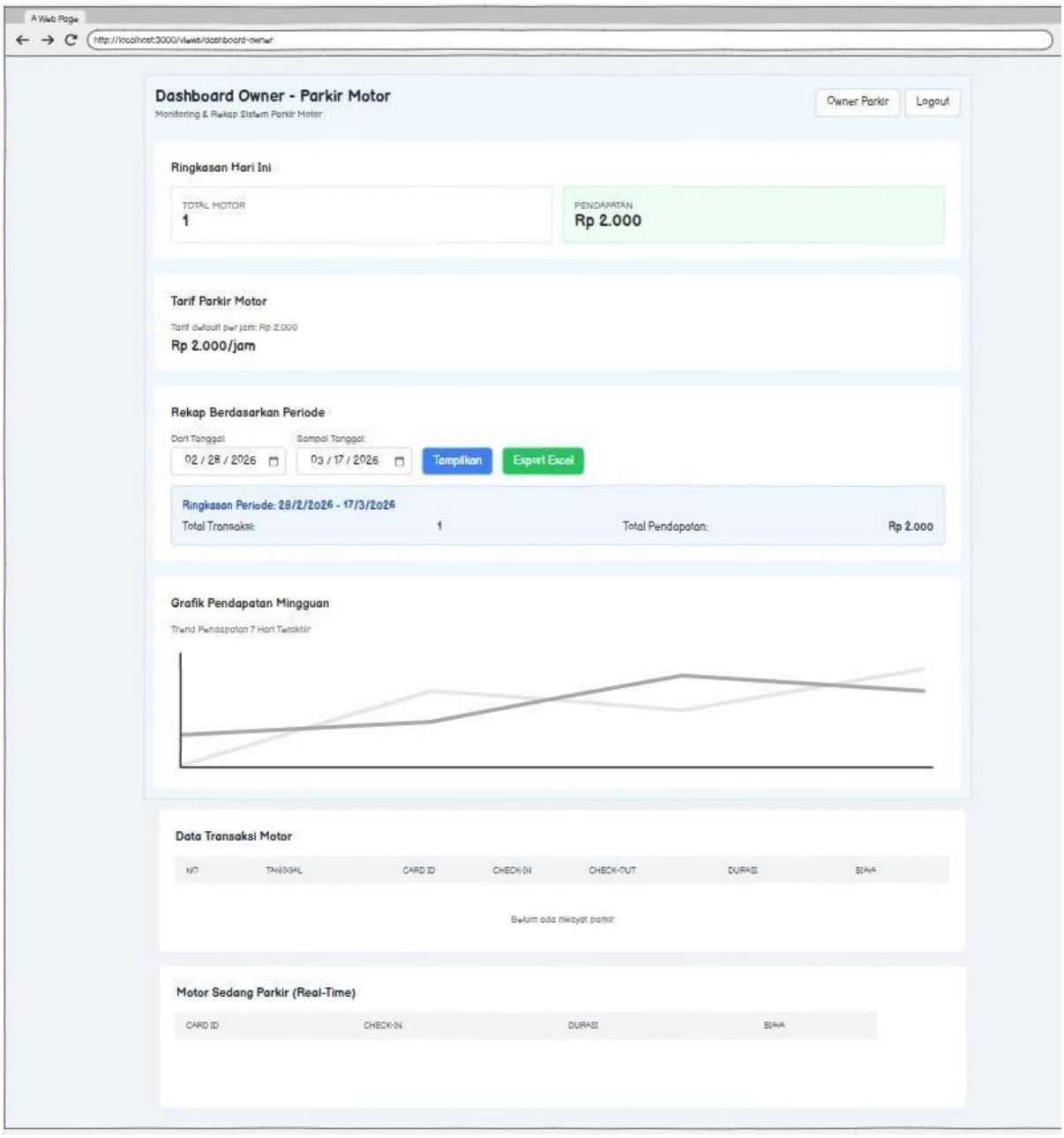
Gambar 8.2 Mockup Halaman Login

Mockup Halaman Utama Dashboard Petugas (Check-In & Check-Out)



Gambar 8.3 Mockup Halaman Utama Dashboard Petugas

Mockup Halaman Utama Dashboard Owner



Gambar 8.4 Mockup Halaman Utama Dashboard Owner

9. UJI COBA

Pengujian Check-In Kendaraan

No	Skenario Pengujian	Test Case	Hasil yang Diharapkan	Hasil Pengujian	Kesimpulan
1	Kendaraan scan RFID di gerbang Entry, belum parkir.	card_id: A1B2C3D4 (belum parkir)	Sistem menyimpan data check-in (status=IN), membuka palang Entry, LCD menampilkan 'Selamat Datang Silakan Masuk'.	Sesuai harapan	Valid
2	Kendaraan scan RFID di gerbang Entry, sudah dalam kondisi parkir (status=IN).	card_id: E5F6G7H8 (status=IN)	Sistem menolak check-in, LCD menampilkan 'Kendaraan Sudah Parkir', palang tidak terbuka.	Sesuai harapan	Valid

Pengujian Check-Out Kendaraan

1	Kendaraan scan RFID di gerbang Exit, kendaraan sedang parkir (status=IN).	card_id: A1B2C3D4 (status=IN)	Sistem menghitung durasi dan biaya, memperbarui status menjadi OUT, menampilkan total biaya di LCD, menunggu konfirmasi petugas.	Sesuai harapan	Valid
2	Kendaraan scan RFID di gerbang Exit, kendaraan tidak sedang parkir.	card_id: I9J0K1L2 (tidak parkir)	Sistem menolak checkout, LCD menampilkan 'Kendaraan Tidak Ditemukan', palang tidak terbuka.	Sesuai harapan	Valid

Pengujian Buka Palang (Konfirmasi Pembayaran)

1	Petugas menekan tombol 'Buka Palang' pada kendaraan status OUT di dashboard.	card_id: A1B2C3D4 (status=OUT)	Palang Exit terbuka via MQTT, LCD menampilkan 'Terima Kasih Selamat Jalan', status berubah menjadi DONE.	Sesuai harapan	Valid
2	Petugas menekan tombol aksi pada kendaraan status IN (bukan OUT).	card_id: E5F6G7H8 (status=IN)	Sistem menolak, tombol aksi tidak tersedia untuk status selain OUT.	Sesuai harapan	Valid

Pengujian Tampilan Dashboard

No	Skenario Pengujian	Test Case	Hasil yang Diharapkan	Hasil Pengujian	Kesimpulan
1	Petugas membuka halaman utama dashboard.	Mengakses URL dashboard melalui browser.	Dashboard menampilkan tiga bagian: daftar CheckIn (status=IN), daftar Check-Out (status=OUT) dengan tombol aksi, dan	Sesuai harapan	Valid
			log riwayat (status=DONE).		
2	Petugas melihat log riwayat transaksi.	Melihat bagian Log Riwayat pada dashboard.	Sistem menampilkan seluruh transaksi status DONE beserta detail card_id, waktu masuk, waktu keluar, durasi, dan biaya.	Sesuai harapan	Valid

10. DESKRIPSI PROGRAM DAN DOKUMENTASI FUNGSI

Sistem Parkir Pintar Berbasis IoT (MyParkir) adalah aplikasi parkir motor yang menghubungkan perangkat keras ESP32 dengan dashboard web secara real-time menggunakan protokol MQTT. Sistem terdiri dari dua komponen utama: **device** (firmware Arduino pada ESP32) dan **dashboard** (aplikasi web berbasis Vanilla JavaScript dengan pola arsitektur MVC: Model–View–Controller). Database dikelola menggunakan Supabase (PostgreSQL) sebagai backend-as-a-service yang menyediakan autentikasi pengguna dan penyimpanan data transaksi.

Terdapat dua role pengguna: **Petugas** yang mengelola operasional parkir harian (memantau kendaraan masuk/keluar dan membuka palang), dan **Owner** yang memantau laporan rekap transaksi dan pendapatan. Komunikasi antara device ESP32 dan dashboard berlangsung melalui broker MQTT `broker.hivemq.com` menggunakan lima topik: `entry/rfid`, `exit/rfid`, `entry/servo`, `exit/servo`, dan `lcd`.

Alur kerja utama sistem adalah sebagai berikut: saat kendaraan tiba di gerbang **Entry**, pengguna melakukan scan kartu RFID—ESP32 membaca UID kartu, mempublikasikan data ke topik MQTT, dashboard menerima dan menyimpan transaksi baru dengan status `IN`, lalu servo membuka palang dan OLED menampilkan pesan sambutan. Saat kendaraan keluar melalui gerbang **Exit**, pengguna scan ulang—sistem menghitung durasi dan biaya, mengubah status menjadi `OUT`, OLED menampilkan total biaya. Setelah pengguna membayar ke petugas, petugas menekan tombol **Buka Palang** di dashboard—status berubah menjadi `DONE` dan palang Exit terbuka.

Berikut adalah dokumentasi fungsi/prosedur utama pada setiap modul aplikasi dashboard Sistem Parkir Pintar Berbasis IoT, disertai penjelasan input, proses, dan output masing-masing.

A. Modul: transaksi-model.js

Fungsi: checkIn(cardId)	
Input	cardId (string) — ID RFID dari kendaraan yang akan parkir.
Proses	1. Mengecek apakah card_id sudah parkir (status=IN) di database Supabase. 2. Jika belum parkir, menyimpan data transaksi baru dengan checkin_time saat ini dan status=IN. 3. Jika sudah parkir, mengembalikan kode error ALREADY_PARKED.
Output	{ success: true, data } jika berhasil, atau { success: false, error, code: 'ALREADY_PARKED' } jika gagal.

Fungsi: handleRFIDExit(payload) — Check-Out	
Input	payload.rfid (string) — card_id dari pesan MQTT gerbang Exit.
Proses	1. Mencari transaksi aktif (status=IN) berdasarkan card_id. 2. Jika tidak ditemukan, mengirim pesan 'DITOLAK Tidak Parkir' ke OLED. 3. Jika ditemukan, menghitung durasi (menit) dan biaya (Rp 2.000/jam dibulatkan ke atas). 4. Memperbarui transaksi: checkout_time, duration, fee, status=OUT di Supabase. 5. Mengirim biaya ke OLED via MQTT.
Output	Data transaksi diperbarui di Supabase (status=OUT). Pesan biaya dikirim ke OLED. Dashboard diperbarui secara otomatis.

Fungsi: handleBukaPalang(transaksiId)	
Input	transaksiId (integer) — ID transaksi yang akan dikonfirmasi keluar oleh petugas.
Proses	1. Memperbarui status transaksi menjadi DONE dan mencatat nama petugas di Supabase. 2. Menerbitkan perintah 'open' ke topik MQTT servo Exit (parking/{nama}/exit/servo). 3. Menerbitkan pesan 'Terima Kasih Selamat Jalan' ke topik MQTT LCD (parking/{nama}/lcd). 4. Memuat ulang data dashboard.
Output	Status transaksi berubah menjadi DONE. Palang Exit terbuka (servo menerima perintah via MQTT). OLED menampilkan pesan perpisahan. Notifikasi sukses ditampilkan di dashboard.

Potongan Kode — transaksi-model.js

```
static async checkIn(cardId, petugasNama) {
  const existing = await this.getByCardId(cardId)
  if (existing.data) return { success: false, code: "ALREADY_PARKED"
}
  const { data } = await supabase.from("transaksi")
    .insert({ card_id: cardId, checkin_time: new
Date().toISOString(),
              status: "IN", petugas_masuk: petugasNama
}).select().single()
  return { success: true, data }
}

static async checkOut(cardId, checkoutTime, duration, fee) {
  const existing = await this.getByCardId(cardId)
  if (!existing.data) return { success: false, code: "NOT_PARKED" }
  const { data } = await supabase.from("transaksi")
    .update({ checkout_time: checkoutTime, duration, fee, status:
"OUT" })
    .eq("id", existing.data.id).select().single()
  return { success: true, data }
}

static async konfirmasiKeluar(id, petugasNama) {
  const { data } = await supabase.from("transaksi")
    .update({ status: "DONE", petugas_keluar: petugasNama })
    .eq("id", id).select().single()
  return { success: true, data }
}
```

B. Modul: utils/helpers.js

Fungsi: hitungBiaya(durasiMenit, tarif)	
Input	durasiMenit (integer) — durasi parkir dalam menit. tarif (integer) — tarif per jam dalam rupiah (default Rp 2.000).
Proses	Mengkonversi durasi menit ke jam dengan pembulatan ke atas (Math.ceil), lalu mengalikan dengan tarif per jam. Minimum 1 jam.
Output	Integer — total biaya parkir dalam rupiah. Contoh: 90 menit → 2 jam → Rp 4.000.

Potongan Kode — utils/helpers.js

```
export function hitungDurasi(waktuMasuk, waktuKeluar = new Date()) {
  return Math.floor((new Date(waktuKeluar) - new Date(waktuMasuk)) /
60000)
}

export function hitungBiaya(durasiMenit, tarifPerJam) {
  return Math.ceil(durasiMenit / 60) * tarifPerJam
}

export function formatRupiah(angka) {
  return "Rp " + angka.toLocaleString("id-ID")
}
```

C. Modul: owner-controller.js

Fungsi: filterData() — Rekap Transaksi Berdasarkan Waktu	
Input	tanggalDari dan tanggalSampai dari input form HTML di dashboard owner.
Proses	1. Memvalidasi rentang tanggal (dari tidak boleh lebih besar dari sampai). 2. Memanggil TransaksiModel.getStatsByDateRange() untuk mengambil data dari Supabase. 3. Menghitung total transaksi dan total pendapatan menggunakan fungsi sumBiaya(). 4. Memperbarui tampilan tabel dan ringkasan di dashboard.
Output	Tabel transaksi, ringkasan total transaksi dan total pendapatan ditampilkan sesuai rentang tanggal yang dipilih.

Potongan Kode — controllers/owner-controller.js

```
async filterData() {
  const dateFrom = new
Date(document.getElementById("tanggalDari").value)
  const dateTo   = new
Date(document.getElementById("tanggalSampai").value)
  dateFrom.setHours(0,0,0,0); dateTo.setHours(23,59,59,999)
  if (dateFrom > dateTo) { showError("Tanggal tidak valid"); return
}
  const result = await TransaksiModel.getStatsByDateRange(
    dateFrom.toISOString(), dateTo.toISOString())
  this.currentData = result.data
  this.updateTransaksiTable()
}

async handleCetakStruk(transaksiId) {
  const { data: t } = await TransaksiModel.getById(transaksiId)
  const struk = window.open("", "_blank", "width=400,height=600")
  struk.document.write(/* HTML struk parkir */)
  struk.document.close()
}
```

D. Modul: petugas-controller.js

Fungsi: handleCetakStruk(transaksiId) — Cetak Struk Parkir	
Input	transaksiId (integer) — ID transaksi yang akan dicetak struknya.
Proses	1. Mengambil data transaksi dari Supabase berdasarkan ID. 2. Membuat window baru (popup) berisi HTML struk parkir dengan format cetak. 3. Auto-print struk saat window terbuka menggunakan window.print().
Output	Window popup berisi struk parkir (card_id, waktu masuk, waktu keluar, durasi, total biaya) yang langsung membuka dialog print browser.

Potongan Kode — controllers/petugas-controller.js

```
async handleRFIDEntry(payload) {
  const result = await TransaksiModel.checkIn(payload.rfid,
this.profile.nama)
  if (!result.success) {
    mqttController.publishLCD("DITOLAK!", "Sudah Parkir"); return
  }
  mqttController.publishServo("entry", "open")
  mqttController.publishLCD("Selamat Datang", "Silakan Masuk")
  await this.loadData()
}

async handleRFIDExit(payload) {
  const trx = (await TransaksiModel.getByCardId(payload.rfid)).data
  if (!trx) { mqttController.publishLCD("DITOLAK!", "Tidak Parkir");
return }
  const dur = hitungDurasi(trx.checkin_time)
  const fee = hitungBiaya(dur, 2000)
  await TransaksiModel.checkOut(payload.rfid, new
Date().toISOString(), dur, fee)
  mqttController.publishLCD("Biaya:"+formatRupiah(fee), "Bayar ke
Petugas")
  await this.loadData()
}

async handleBukaPalang(transaksiId) {
  await TransaksiModel.konfirmasiKeluar(transaksiId,
this.profile.nama)
  mqttController.publishServo("exit", "open")
  mqttController.publishLCD("Terima Kasih", "Selamat Jalan")
  await this.loadData()
}
```

E. Modul: sketch/myparkirgua.ino (Firmware ESP32)

Fungsi: readRFID(rfid)	
Input	rfid (MFRC522&) — referensi objek RFID reader (Entry atau Exit).
Proses	Memeriksa apakah ada kartu baru di depan reader. Jika ada, membaca UID byte per byte, mengkonversi ke String HEX kapital, kemudian menghentikan komunikasi dengan kartu (PICC_HaltA dan PCD_StopCrypto1).
Output	String UID kartu dalam format HEX kapital (contoh: "A1B2C3D4"), atau String kosong "" jika tidak ada kartu terdeteksi.

Fungsi: openServo(servo)	
Input	servo (Servo&) — referensi objek servo yang akan digerakkan (servoEntry atau servoExit).
Proses	Menggerakkan servo ke posisi 90 derajat (palang terbuka), menunggu 3 detik (delay 3000ms), kemudian mengembalikan servo ke posisi 0 derajat (palang tertutup).
Output	Palang gerbang terbuka selama 3 detik kemudian menutup kembali secara otomatis.

Fungsi: showOLED(line1, line2)	
Input	line1 (String) — teks baris pertama. line2 (String) — teks baris kedua yang ditampilkan di OLED.
Proses	Membersihkan layar OLED (clearDisplay), mengatur ukuran dan warna teks, kemudian mencetak line1 di posisi y=10 dan line2 di posisi y=40, lalu memanggil display() untuk menampilkan hasilnya.
Output	Layar OLED 128x64 menampilkan dua baris teks pesan kepada pengguna (contoh: "Selamat Datang" / "Silakan Masuk").

Fungsi: callback(topic, payload, length) — MQTT Message Handler	
Input	topic (char*) — nama topik MQTT. payload (byte*) — isi pesan. length (unsigned int) — panjang pesan.
Proses	Mengkonversi payload byte ke String. Memeriksa topik: (1) jika topik Entry Servo dan pesan mengandung "OPEN" → panggil openServo(servoEntry). (2) jika topik Exit Servo dan pesan mengandung "OPEN" → panggil openServo(servoExit). (3) jika topik LCD → pisahkan pesan berdasarkan separator "|" dan panggil showOLED(line1, line2).
Output	Servo Entry/Exit bergerak membuka palang, atau OLED menampilkan pesan sesuai instruksi dari dashboard melalui MQTT.

Potongan Kode — sketch/myparkirgua.ino

```
String readRFID(MFRC522 &rfid) {
  if (!rfid.PICC_IsNewCardPresent()||!rfid.PICC_ReadCardSerial())
return "";
  String uid = "";
  for (byte i=0; i<rfid.uid.size; i++) uid +=
String(rfid.uid.uidByte[i], HEX);
  uid.toUpperCase(); rfid.PICC_HaltA(); rfid.PCD_StopCrypto1();
  return uid;
}

void openServo(Servo &servo) { servo.write(0); delay(3000);
servo.write(90); }

void showOLED(String line1, String line2) {
  display.clearDisplay(); display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0,10); display.println(line1);
  display.setCursor(0,40); display.println(line2);
  display.display();
}

void callback(char *topic, byte *payload, unsigned int length) {
  String msg = "";
  for (int i=0; i<length; i++) msg += (char)payload[i];
  if (String(topic)==TOPIC_ENTRY_SERVO&&msg.indexOf("OPEN")>=0)
openServo(servoEntry);
  if (String(topic)==TOPIC_EXIT_SERVO &&msg.indexOf("OPEN")>=0)
openServo(servoExit);
  if (String(topic)==TOPIC_LCD) {
    int sep=msg.indexOf("|");
    showOLED(msg.substring(0,sep), msg.substring(sep+1));
  }
}
```

11. LAPORAN EVALUASI SINGKAT

A. Fitur yang Sudah Berjalan dengan Baik

1. Check-In kendaraan via scan RFID di gerbang Entry — data tersimpan ke Supabase secara real-time, palang Entry terbuka otomatis via MQTT, OLED menampilkan pesan sambutan.
2. Check-Out kendaraan via scan RFID di gerbang Exit — durasi dihitung otomatis, biaya dibulatkan per jam (Rp 2.000/jam), status berubah menjadi OUT, total biaya tampil di OLED.
3. Konfirmasi buka palang oleh petugas melalui tombol aksi di dashboard — status transaksi berubah menjadi DONE, palang Exit terbuka, OLED menampilkan pesan perpisahan.
4. Dashboard petugas menampilkan tiga bagian secara real-time: daftar kendaraan parkir aktif (status=IN), daftar kendaraan akan keluar (status=OUT), dan log riwayat transaksi selesai (status=DONE).
5. Cetak struk parkir — petugas dapat mencetak struk berisi detail transaksi langsung dari browser.
6. Rekap transaksi berdasarkan rentang waktu di dashboard owner — menampilkan total kendaraan dan total pendapatan pada periode yang dipilih, dilengkapi fitur export ke Excel.
7. Sistem multi-role (Petugas dan Owner) — tampilan dashboard berbeda sesuai role pengguna yang login.
8. Koneksi MQTT via WebSocket ke broker.hivemq.com:1883 — komunikasi dua arah antara device ESP32 dan dashboard berjalan stabil.

B. Bug yang Belum Diperbaiki

1. Koneksi MQTT dapat terputus saat broker mengalami gangguan — belum ada mekanisme reconnect otomatis yang stabil; saat ini pengguna perlu me-refresh halaman secara manual.
2. Validasi card_id hanya berdasarkan status aktif di database — belum ada whitelist card_id terdaftar, sehingga kartu RFID yang tidak dikenal tetap bisa melakukan check-in.
3. Tampilan dashboard belum responsif penuh pada layar berukuran kecil (mobile) — beberapa tabel meluber pada resolusi di bawah 768px.

C. Rencana Pengembangan Berikutnya

1. Menambahkan fitur CRUD Kendaraan — hanya card_id terdaftar yang dapat melakukan check-in, dilengkapi manajemen data pemilik kendaraan.
2. Menambahkan fitur CRUD Tarif Parkir — tarif dapat diatur secara dinamis melalui dashboard tanpa perlu mengubah kode program.
3. Mengimplementasikan fitur CRUD Area Parkir — sistem menolak check-in otomatis saat kapasitas parkir sudah penuh.
4. Menambahkan mekanisme reconnect MQTT otomatis agar koneksi antara device dan dashboard lebih stabil tanpa perlu refresh halaman.
5. Mengembangkan tampilan dashboard menjadi fully responsive agar dapat diakses dengan nyaman dari perangkat mobile.

12. DEBUGGING

Berikut adalah dokumentasi error dan bug yang ditemukan selama proses pengembangan Sistem Parkir Pintar Berbasis IoT beserta solusi yang diterapkan:

No	Error / Bug	Penyebab	Solusi	Status
1	ESP32 gagal terhubung ke broker MQTT (connection refused)	Client ID yang digunakan tidak unik sehingga broker menolak koneksi karena ID sudah dipakai device lain.	Membuat fungsi getMqttClientId() yang menggunakan MAC address chip ESP32 (getEfuseMac) agar Client ID selalu unik.	Selesai
2	OLED tidak menampilkan teks (layar tetap gelap setelah inisialisasi)	Pin SDA/SCL default Wire (GPIO21/22) belum dideklarasikan secara eksplisit sebelum memanggil display.begin().	Menambahkan Wire.begin(21, 22) secara eksplisit sebelum inisialisasi OLED di fungsi setup().	Selesai
3	RFID membaca kartu lebih dari sekali dalam satu scan (double publish ke MQTT)	Tidak ada delay setelah publish sehingga dalam satu kali tempel kartu, loop() berjalan beberapa kali dan membaca kartu yang sama berulang.	Menambahkan delay(2000) setelah setiap publish RFID di fungsi loop() agar ada jeda 2 detik sebelum pembacaan berikutnya.	Selesai
4	Data transaksi tidak tersimpan ke Supabase (error 400 Bad Request)	Nama kolom di kode JavaScript tidak sesuai dengan nama kolom di tabel Supabase (perbedaan huruf kapital dan underscore).	Menyamakan penulisan nama kolom di kode dengan skema tabel di Supabase (checkin_time, checkout_time, card_id, dll) menggunakan huruf kecil dan underscore.	Selesai
5	Dashboard tidak update otomatis setelah kendaraan check-in/check-out (harus refresh manual)	Fungsi loadData() tidak dipanggil ulang setelah menerima pesan MQTT masuk dari device.	Menambahkan pemanggilan await this.loadData() di akhir setiap handler RFID (handleRFIDEntry dan handleRFIDExit) dan menambahkan auto-refresh interval setiap 10 detik.	Selesai